Ms08067安全实验室-红队攻防实战培训班 第5期

#####学习更多红队实战技术，欢迎加入我们，详细授课大纲见公众号介绍（二维码在文章底部）

#

关于讲课内容：直接就讲关键部分，关于基础部分的内容大家下来自己学习或者问我

关于就业：就业需要哪些东西，需要学习哪些东西

1.基础知识

web搭建

osi 7层模型

常见漏洞原理 sql注入 ssrf

平时可以复现一些漏洞 struts2 weblogic shiro fastjson

2.实战经验：

  1.src

  2.hw

  3.审计

3.工具：

  知道工具的底层原理

  尝试编写自己的工具

4.文章：

  博客

  各大安全网站发文章

  b站

5.参加会议：

  发言

  组织

关于被动信息收集中所遇到的问题：

被动信息收集怎么去思考还有哪些工具或者渠道可以使用（主要是靠积累）

记住结构

搜索引擎

github

综合性工具

很多工具肯定会存在信息的重叠，这个时候就要做比较，怎么做：看你的需求：1.速度快 2.信息多

被动信息收集中收集到的信息有没有相应的模板

不限制

要求：全面 准确 逻辑清晰

被动信息收集中经常不够耐心细心怎么办

要有这个概念：

信息收集这个流程是占用时间最长的

对于收集到的信息一定要多分析 多看

实战中的被动信息收集都是怎么去做的

我们会有2个部分做终端：

第三方平台

数据提供商

服务器-----就是去查询搜索引擎------重点我们会查询和公司新闻相关

综合信息

做二次处理

先让机器做二次处理：正则、nlp

然后再人工检查

eg：

公司：1000个ip

熟悉对方业务

边缘业务：200ip

新开的业务：50ip

主动信息收集

1.主动信息收集的基本结构

基本流程：

1.由攻击者主动发起行为或请求

2.目标受到行为或请求影响并发出反馈

3.攻击者获得反馈并判断特征

2.主动信息收集的特点和应用场景

主动信息收集的优缺点：

优点：

1.信息的针对性强

2.信息的准确度和时效性高

3.能收集到未公开的敏感信息

缺点：

1.容易暴露，被发现的风险高

2.收集的信息量和覆盖面比较少

主动信息收集的应用场景：

主动信息收集通常是在**被动信息收集之后分析出重要攻击面或者目标重要信息的时候再针对性地去做主动信息收集**，在主动信息收集的时候通常持续时间比较长而且会通过各种不同的方式与目标交互。**实战中最常用的手段是模拟或者伪装正常的业务流程与目标交互。**

绕过cdn的常见方法：

cdn：通过不同地理位置的缓存来加快访问速度，所以我们有可能访问的网页不是真实ip提供的网页

思路：

cdn是要钱的

cdn是基于解析的

1.dns历史记录 找最早的那一条

以下几个网站来查询。

https://x.threatbook.cn

https://webiplookup.com

https://viewdns.info/iphistory

https://securitytrails.com/#search

https://toolbar.netcraft.com/site_report

举个栗子：www.gwfutures.cn

2.多地ping/国外ping

3.边缘业务的子域名

微信小程序

app客户端

4.证书

https://search.censys.io/

https://crt.sh/

5.邮箱服务器

```
cdn是不支持邮箱的
```

eg：ping www.beijing.gov.cn

ping mail.beijing.gov.cn

6.FOFA查询网站标题找到真实IP

3.主动信息收集的常见手段

3.1 通过暴力枚举的方法寻找目标存在的资产

基本原理：

1.工具通常带有字典等枚举的数据（字典自己写的时候该注意哪些东西：构造完成的请求中哪些是不变的，哪些是字典里的）

2.通过每一个枚举的数据构造请求或者交互（dns、http、ip、密码、参数）

3.判断这些请求中哪些交互的反馈是属于我们需要的（每一个对象的反馈特征）

4.存储这些反馈并处理数据

以ksubdomian举例：

```
ksubdomain -d jd.com -o jd.com.txt
```

返回的结果如下：

```
C:\Users\muDONG\Desktop\1\lession2\ksubdomain>ksubdomain.exe -d baidu.com -o baidu.txt


 _  __   _____       _         _                       _
| |/ /  / ____|     | |       | |                     (_)
| ' /  | (___  _   _| |__   __| | ___  _ __ ___   __ _ _ _ __
|  <    \___ \| | | | '_ \ / _| |/ _ \| '_   _ \ / _  | | '_ \
| . \   ____) | |_| | |_) | (_| | (_) | | | | | | (_| | | | | |
|_|\_\ |_____/ \__,_|_.__/ \__,_|\___/|_| |_| |_|\__,_|_|_| |_|

[INFO] Current Version: 0.7
[INFO] Npcap version 1.00, based on libpcap version 1.9.1
...
[INFO] Use Device: \Device\NPF_{D2D4F1C7-EC94-4D0A-B5F1-50E0287AACCB}
[INFO] Use IP:192.168.1.4
[INFO] Local Mac:80:2b:f9:32:5d:46
[INFO] GateWay Mac:30:50:fd:68:bd:74
[INFO] 加载内置字典
[INFO] 检测域名:[baidu.com]
[INFO] 设置rate:10000pps
[INFO] DNS:[223.5.5.5 223.6.6.6 180.76.76.76 119.29.29.29 182.254.116.116
114.114.114.115]
insurance.baidu.com => CNAME news.n.shifen.com => 180.101.49.131
qh.baidu.com => 10.46.139.100
ns8.baidu.com => 111.206.208.152
zhanzhang.baidu.com => CNAME zhanzhang.n.shifen.com => 182.61.201.91 => 182.61.201.90
fff.baidu.com => CNAME ff.n.shifen.com => 220.181.43.9
```

```
cmp.baidu.com => 10.242.76.34
doctor.baidu.com => CNAME fengchao-new.e.shifen.com => 220.181.33.196 =>
124.237.176.116
rpc.baidu.com => CNAME orphttps.n.shifen.com
xf.baidu.com => CNAME mrp4.n.shifen.com => 220.181.163.54
kst.baidu.com => 10.100.47.127
hd.baidu.com => CNAME bapp.n.shifen.com => 180.149.131.33
cb.baidu.com => CNAME cb.e.shifen.com => 220.181.107.139
dcc.baidu.com => 120.48.7.36
lmp.baidu.com => 180.97.33.24 => 111.206.37.67
cpu.baidu.com => CNAME cpu.e.shifen.com => 180.101.49.208
tuijian.baidu.com => CNAME tuijian.hm.e.shifen.com => 220.181.33.173
crab.baidu.com => CNAME crab.n.shifen.com => 220.181.107.178
wm.baidu.com => CNAME wangmeng.e.shifen.com => 115.239.210.100 => 115.239.210.106
mission.baidu.com => CNAME 730.nuomi.n.shifen.com => 180.97.34.232 => 180.97.93.28
j.baidu.com => CNAME js.hm.e.shifen.com => 39.156.66.184 => 39.156.66.185
pmp.baidu.com => CNAME pheonest.e.shifen.com => 124.237.176.206
pgc.baidu.com => CNAME sv.map.n.shifen.com => 220.181.43.133
lady.baidu.com => CNAME news.baidu.com => CNAME news.n.shifen.com => 180.101.49.131
www6.baidu.com => CNAME www6.a.shifen.com => 110.242.68.4 => 110.242.68.3
tixing.baidu.com => 10.215.7.192
hm.baidu.com => CNAME hm.e.shifen.com => 124.237.176.160
oops.baidu.com => CNAME m1-psop-con0.m1.baidu.com
yb.baidu.com => CNAME app.video.n.shifen.com => 180.76.174.35 => 106.12.156.239 =>
180.76.156.35 => 106.12.165.163
byp.baidu.com => CNAME shopc.e.shifen.com => 124.237.177.2
```

基本原理：通过枚举每一个字母的子域名去查询cname解析记录，如果dns的cname解析记录存在则返回结果

nslookup -q=cname xx.baidu.com

ping xx.baidu.com

以dirsearch举例：

```
C:\Users\muDONG\Desktop\1\lession2\dirsearch-0.4.0>python3 dirsearch.py -u
www.baidu.com -e *

  _|. _ _  _  _  _ _|_    v0.3.9
 (_||| _) (/_(_|| (_| )

Extensions: * | HTTP method: GET | Threads: 20 | Wordlist size: 6784

Error Log: C:\Users\muDONG\Desktop\1\lession2\dirsearch-0.4.0\logs\errors-21-11-06_11-
56-04.log

Target: www.baidu.com
```

```
Output File: C:\Users\muDONG\Desktop\1\lession2\dirsearch-
0.4.0\reports\www.baidu.com\_21-11-06_11-56-04.txt

[11:56:05] Starting:
[11:56:06] 302 -  231B  - /*.php  ->  http://www.baidu.com/forbiddenip/forbidden.html
[11:56:06] 302 -  231B  - /.atoum.php  ->
http://www.baidu.com/forbiddenip/forbidden.html
[11:56:07] 302 -  231B  - /.configuration.php  ->
http://www.baidu.com/forbiddenip/forbidden.html
[11:56:09] 302 -  231B  - /.htaccess  ->
http://www.baidu.com/forbiddenip/forbidden.html
[11:56:09] 302 -  231B  - /.htaccess-dev  ->
http://www.baidu.com/forbiddenip/forbidden.html
[11:56:09] 302 -  231B  - /.htaccess-local  ->
http://www.baidu.com/forbiddenip/forbidden.html
[11:56:09] 302 -  231B  - /.htaccess-marco  ->
http://www.baidu.com/forbiddenip/forbidden.html
[11:56:09] 302 -  231B  - /.htaccess.bak1  ->
http://www.baidu.com/forbiddenip/forbidden.html
[11:56:09] 302 -  231B  - /.htaccess.bak  ->
http://www.baidu.com/forbiddenip/forbidden.html
```

基本原理：通过枚举字典中的每一个数据去请求http并判断返回值，返回值为200则表示该目录存在

```python
import requests


if __name__ == "__main__":
    url = input("输入url:")
    with open(".\zidian.txt" ) as f:  # 打开文件
        for line1 in f:
            re = requests.get(url + "/" + line1)
            if re.status_code == 200:
                print(url + "/" + line1 + "  is ok")
            else:
                pass
```

3.2 通过爬虫爬取的方式收集信息

基本原理：

1.工具会模拟浏览器做请求

2.将收到的请求收集并筛选（筛选什么东西：链接、按钮、只能能跳转的）

3.筛选出来的数据做二次的请求并判断

4.将判断后的数据处理并输出

以jsfinder举例：

```
C:\Users\muDONG\Desktop\1\lession2\JSFinder-master\JSFinder-master>python3 JSFinder.py
-u http://www.baidu.com
url:http://www.baidu.com
Find 85 URL:
http://nsclick.baidu.com
https://www.baidu.com/nocache/fesplg/s.gif?log_type=sp
https://www.baidu.com/s?wd=
https://www.baidu.com/nocache/fesplg/s.gif?log_type=hm&type=ssl&
https://gt1.baidu.com/nocache/imgdata/sp613.gif?t=
https://gt2.baidu.com/nocache/imgdata/sp613.gif?t=
https://www.baidu.com
https://www.baidu.com/s?
https://www.baidu.com/nocache/fesplg/s.gif?log_type=linksp
http://www.baidu.com
http://bzclk.baidu.com
https://sp0.baidu.com/9q9JcDHa2gU2pMbgoY3K
http://v.baidu.com
https://www.baidu.com/nocache/fesplg/s.gif?log_type=hm
https://www.baidu.com/cache/user/html/v3Jump.html
https://www.baidu.com/cache/user/html/xd.html
http://shadu.baidu.com/landingpage/competing.html?from=10064
https://www.baidu.com/cache/aladdin/ui/
http://opendata.baidu.com/api.php
http://open.baidu.com/stat/al_e.gif?ajax_err_url=#{url}
https://passport.baidu.com/passApi/js/uni_login_wrapper.js?cdnversion=
https://www.baidu.com/my/index
https://mbd.baidu.com/newspage/api/getttsurllist
http://changrui.bcc-bdbl.baidu.com:8813/report/report_audio_land_page
https://www.baidu.com/s?rsv_dl=selectedsearch&wd=
http://nourl.ubs.baidu.com
https://www.baidu.com/
http://isphijack.baidu.com/index.php?cb=isp_hijack
https://www.baidu.com/tools
http://i.baidu.com
http://bdimg.share.baidu.com/static/api/js/custom/resultshare.js
http://jubao.baidu.com
http://koubei.baidu.com
http://j.br.baidu.com/v1/t/ui/p/browser/tn/10105001/ch_dl_url
http://tag.baidu.com
http://s.share.baidu.com
http://bdimg.share.baidu.com
http://s.share.baidu.com/?
```

```
https://www.baidu.com/tools?url=
https://www.baidu.com/cache/fpid/ielib_0108.js
https://www.baidu.com/cache/fpid/chromelib_0108.js
http://eclick.baidu.com/ps_fp.htm?
http://sestat.baidu.com
https://ss0.baidu.com/6ONWsjip0QIZ8tyhnq/ps_default.gif
https://ss1.baidu.com/6ONWsjip0QIZ8tyhnq/ps_default.gif
https://ss2.baidu.com/6ONWsjip0QIZ8tyhnq/ps_default.gif
https://ss3.baidu.com/6ONWsjip0QIZ8tyhnq/ps_default.gif
https://www.baidu.com/con?from=self
https://www.baidu.com/ups/submit/addtips/?product=ps&tips=
https://www.baidu.com/r/plog
http://velocity.baidu.com/sp
https://sp0.baidu.com/6r1_czmhAB63otqbppnN2DJv/sp
https://www.baidu.com/?tn=baidu
https://www.baidu.com/nocache/fesplg/s.gif?log_type=hm&type=timing&
https://pcrec.baidu.com/link?url=
https://www.baidu.com/nocache/fesplg/s.gif?lid=
https://m.baidu.com/logo.gif
https://ss0.baidu.com/5bd1bjqh_Q23odCf/static/wiseindex/img/w_icon2.png?ran
http://sestat.baidu.com/cm.gif?type=cdnmonitor
https://sptidchk.baidu.com/s.gif
https://sptidcsfo.baidu.com/s.gif
https://sptidcjp.baidu.com/s.gif
https://sptidcsin.baidu.com/s.gif
http://sensearch.baidu.com/sensearch/selecttext
http://ufo.baidu.com/listen/myhistory?product_line=20018&appid=215622&type=commonQA
http://ufo.baidu.com/listen/myhistory?type=myhistory&product_line=20018&appid=215622
https://wappass.baidu.com/static/machine/js/api/mkd.js
http://f3.baidu.com
https://sp1.baidu.com/5b1ZeDe5KgQFm2e88IuM_a/webb.gif
http://sestat.baidu.com/webb.gif
https://sp1.baidu.com/5b1ZeDe5KgQFm2e88IuM_a/mwb2.gif
http://sestat.baidu.com/mwb2.gif
https://vse.baidu.com/echo.fcgi
https://www.baidu.com/home/hit/v.gif?
https://m.baidu.com/tc?tcreq4log=1
https://hpd.baidu.com/v.gif
https://www.baidu.com/nocache/fesplg/s.gif?log_type=hm&type=uamonitor&
http://sclick.baidu.com/w.gif?fm=suggestion&title=%B9%D8%B1%D5&t=
http://sclick.baidu.com/w.gif?q=
http://suggestion.baidu.com/su
https://www.baidu.com/cache/fpid/lib_1_0.js
https://www.baidu.com/cache/fpid/ielib_1_1.js
https://www.baidu.com/cache/fpid/chromelib_1_1.js
http://dj0.baidu.com/v.gif?pid=315&type=2011&portrait=
http://sclick.baidu.com/w.gif

Find 45 Subdomain:
```

```
nsclick.baidu.com
www.baidu.com
gt1.baidu.com
gt2.baidu.com
bzclk.baidu.com
sp0.baidu.com
v.baidu.com
shadu.baidu.com
opendata.baidu.com
open.baidu.com
passport.baidu.com
mbd.baidu.com
changrui.bcc-bdbl.baidu.com:8813
nourl.ubs.baidu.com
isphijack.baidu.com
i.baidu.com
bdimg.share.baidu.com
jubao.baidu.com
koubei.baidu.com
j.br.baidu.com
tag.baidu.com
s.share.baidu.com
eclick.baidu.com
sestat.baidu.com
ss0.baidu.com
ss1.baidu.com
ss2.baidu.com
ss3.baidu.com
velocity.baidu.com
pcrec.baidu.com
m.baidu.com
sptidchk.baidu.com
sptidcsfo.baidu.com
sptidcjp.baidu.com
sptidcsin.baidu.com
sensearch.baidu.com
ufo.baidu.com
wappass.baidu.com
f3.baidu.com
sp1.baidu.com
vse.baidu.com
hpd.baidu.com
sclick.baidu.com
suggestion.baidu.com
dj0.baidu.com
```

原理：通过请求页面并获得页面中的js文件，筛选js文件中的子域名，这个工具并没有做判断，直接返回所有的子域名

以zap的spider举例：

基本原理：通过浏览器请求页面的返回并筛选所有的<a>标签并通过模拟浏览器的操作点击网页中的每个动态链接并判断返回值

```
import requests
from bs4 import BeautifulSoup

def qingqiu():
    re = requests.get("mubiao")
    a = re.text
    return a

def shaixuan(a):
    soup = BeautifulSoup("lxml")
    soup.find
```

3.3 通过直接交互的方式收集目标反馈

基本原理：

1.工具会带有直接与目标交互的功能

2.通过判断目标反馈的不同判断出不同的信息

《图解tcpip》

https://www.bilibili.com/video/BV1b7411Z7Ba nmap脚本编写

以nmap举例：

--script 使用脚本

-oN输出普通文本

-oX输出成xml

```
C:\Users\muDONG\Desktop\1\lession2\ksubdomain>nmap -sV 127.0.0.1
Starting Nmap 7.91 ( https://nmap.org ) at 2021-11-06 15:49 ?D1ú±ê×?ê±??
```

```
Nmap scan report for localhost.localdomain (127.0.0.1)
Host is up (0.0012s latency).
Not shown: 994 closed ports
PORT     STATE SERVICE         VERSION
135/tcp  open  msrpc           Microsoft Windows RPC
445/tcp  open  microsoft-ds?
902/tcp  open  ssl/vmware-auth VMware Authentication Daemon 1.10 (Uses VNC, SOAP)
912/tcp  open  vmware-auth     VMware Authentication Daemon 1.0 (Uses VNC, SOAP)
3389/tcp open  ms-wbt-server   Microsoft Terminal Services
5357/tcp open  http            Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.87 seconds
```

pip3 install python-nmap

```
import nmap

nm = nmap.PortScanner()
nm.scan('127.0.0.1','1-100','-sV')
a = nm.scan('127.0.0.1','1-65535','-sV')
print(type(a))
print(a)
```

基本原理：通过发送网络请求包并判断返回值获得信息

综合性的直接探测，以goby举例：

4.如何编写基本的主动信息收集工具

三次握手

发送一个syn

服务器返回一个 syn+ack 不开的话 返回一个rst

以scapy举例

```
# -*- coding: utf-8 -*-
from scapy.all import *
```

```
dst_ip="121.196.151.60"
src_port=RandShort()
dst_port=22

tcp_connect_scan_resp =
sr1(IP(dst=dst_ip)/TCP(sport=src_port,dport=dst_port,flags="S"),timeout=10)
if(str(type(tcp_connect_scan_resp))==""):
  print "Closed"
elif(tcp_connect_scan_resp.haslayer(TCP)):
  if(tcp_connect_scan_resp.getlayer(TCP).flags == 0x12):#0x12(表示“SYN+ACK”)那么表示端口没开
    send_rst =
sr(IP(dst=dst_ip)/TCP(sport=src_port,dport=dst_port,flags="AR"),timeout=10)
    print "Open"
elif (tcp_connect_scan_resp.getlayer(TCP).flags == 0x14):#0x14(表示RST)那么表示端口没开
  print "Closed"
```

os.system()

基本思路：

和POC编写的思路一样分为2个部分：

1.请求的功能

2.判断的功能

5.主动信息收集收集到的信息的处理

5.1 分类

将收集到的信息以资产的个体做分类

5.2 拓扑

将收集到的信息做再一次的拓扑

5.3 整合

将收集到的信息做整合并合并到之前的被动信息收集的结果中

6.实战中的主动信息收集都是怎么做的

分布式，长时间，不连续性的，这三种特点下的主动信息收集并且收集到的信息会和被动信息收集到的信息做匹配和对应处理，最常用的手工方式是模拟或者伪装正常的业务请求获得信息。

7.工具之间的数据流动

上一个工具的输出--->下一个工具的输入

公司名称

1，收集主域名　搜索引擎 备案网站

AppInfoScanner

2，收集子域名　ssl dns记录 搜索引擎 威胁情报网站

3，端口扫描　msscan　nmap　goby

4，指纹识别　whatweb（http://whatweb.bugscaner.com/look/）　Wappalyzer插件　goby　云悉

thinkphp　dedecms　oa　jsp　php　asp　win linux

5，目录扫描 dirsearch　御剑

6，漏洞扫描 awvs　xray　vulmap

```
 针对性的扫描    oa   cms    thinkphp
```

搜集员工信息，招标信息 github等　百度贴吧 新浪微博 手机号 支付宝转账　论坛

目标人员邮箱：

```
# -*- coding: utf-8 -*-
# @Time : 2021年08月04日:14时34分
# @File : snov.py
import requests
import re
import json
from urllib.parse import unquote
from math import ceil
from optparse import OptionParser  #用于生成帮助文档
from os.path import exists
import warnings
warnings.filterwarnings('ignore')

class Snov:
    def __init__(self):
        self.domain="https://app.snov.io"
        self.headers={
```

```
            'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_4)
AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.1 Safari/605.1.15'
        }
        self.session=requests.session()


    def _login(self,username,password):  #自动登录功能
        session=requests.session() #定义session
        url=self.domain+"/login"
        response=session.get(url=url,headers=self.headers)
        data=response.content.decode()
        pattern="name=\"_token\"\s?value=\"(.*?)\"" #获取登录使用的token
        result=re.findall(pattern,data)
        try:
            _token=result[0]
        except:
            return False
        param={
            "email":username,
            "password":password,
            "_token":_token,
            "remember":"off"
        }
        response=session.post(url=url,data=param,headers=self.headers,verify=False)
        XSRF_TOKEN=session.cookies.get("XSRF-TOKEN")
        self.headers['X-XSRF-TOKEN']=unquote(XSRF_TOKEN)  #将XSRF_TOKEN复制
        self.session=session
        return True

    def _getDomainSearch(self,searchDomain):
        param={"domain":searchDomain,"isGreen":"true","lastId":0,"perPage":20}
        url=self.domain+"/domain-search"
        headers = self.headers
        headers['Content-Type'] = "application/json"

response=self.session.post(url=url,headers=headers,data=json.dumps(param),verify=False)
        data=response.content.decode()
        data=json.loads(data)
        try:
            id=data.get("companyInfo",None).get('id',None)
            return id  #返回公司id
        except:
            return False

    def _getTotal(self,id):  #获取邮箱和联系人总个数
        url=self.domain+"/companies/"+str(id)
        headers = self.headers
        headers['Content-Type'] = "application/json"
        response=self.session.post(url=url,data=None,headers=headers,verify=False)
```

```
        data=response.content.decode()
        data=json.loads(data)
        try:
            emailsTotal=data.get("emailsTotal",0)
            contactsTotal=data.get("contactsTotal",0)
            return (emailsTotal,contactsTotal)
        except:
            return (0,0)
    def _getMail(self,total,id,target):  #获取全部邮箱
        url = self.domain + "/companies/" + str(id) + "/" + "emails"
        headers = self.headers
        headers['Content-Type'] = "application/json"
        headers['Referer'] = "https://app.snov.io/domain-search?name=
{}&tab=emails".format(target)
        headers['X-Requested-With'] = "XMLHttpRequest"
        #total=452
        count=ceil(total/200)
        print("{}一个有{}个邮箱，分{}次获取".format(target,total,count))
        lastId=None
        for i in range(1,count+1,1):
            if not lastId:
                param={"isGreen":True,"lastId":0,"perPage":200,"page":i}

response=self.session.post(url=url,data=json.dumps(param),headers=headers,timeout=50,ve
rify=False)
                data=response.content.decode()
                data=json.loads(data)
                lists=data.get("list",None)
                lastId=data.get("lastId",1)
                self._storeMail(lists,target)
            elif lastId:
                param={"isGreen":True,"lastId":lastId,"perPage":200,"domain":""}
                response = self.session.post(url=url, data=json.dumps(param),
headers=headers, timeout=50, verify=False)
                data = response.content.decode()
                data = json.loads(data)
                lists = data.get("list", None)
                lastId = data.get("lastId", 1)
                self._storeMail(lists,target)

    def _storeMail(self,lists,target):
        storage="./"+target+"_mails.txt"
        if lists:
            with open(storage,"a",encoding="utf-8") as fp:
                for list in lists:
                    email=list.get('email',None)
                    if email:
                        fp.write(email)
                        fp.write("\n")
```

```
    def run(self,username,password,targets):
        if self._login(username=username,password=password):
            for target in targets:
                print("开始查询{}。".format(target))
                id=self._getDomainSearch(target.strip())
                emailsTotal, contactsTotal = self._getTotal(id)
                self._getMail(emailsTotal,id,target)
        else:
            exit("登录出错！")



if __name__ == '__main__':
    parser = OptionParser()
    parser.add_option("-u", "--user", dest="username", default=None, help="用户名")
    parser.add_option("-p", "--pass", dest="password", default=None, help="登录密码")
    parser.add_option("-t", "--target", dest="target", default=None, help="要查询的域名，
多个目标使用,隔开")
    parser.add_option("-f", "--file", dest="file", default=None,  help="可以使用文件输入目
标，一行一个")
    (options, args) = parser.parse_args()  # 取值并且存入options
    if (options.username and options.password):
        if options.target:
            targets=options.target.split(",")
            print(targets)
        elif exists(options.file):
            targets=list()
            with open(options.file,"r",encoding='utf-8') as fp:
                line=fp.readline()
                while line:
                    line=line.strip()
                    targets.append(line)
                    line=fp.readline()
        else:
            targets=None
        if targets:
            snov=Snov()
            snov.run(options.username.strip(),options.password.strip(),targets)
    else:
        exit("输入有误，滚啊！！！")
```

凌风云：https://www.lingfengyun.com/

蓝菊花：http://www.lanjuhua.com/

大力盘：https://www.dalipan.com/

猪猪盘：http://www.zhuzhupan.com/

PanSou：http://www.pansou.com/

盘飞飞：https://panfeifei.com/

作业1（选做）（使用任意编程语言编写一个网络空间搜索引擎的子域名搜索结果为输入，主动信息收集工具输出端口的综合性工具）

作业2（对一个目标所开放的业务亲自去过一遍业务流程，然后每一个业务阶段或者业务页面都抓包截图并描述大概情况）

模板：

链接（url）	截图（页面截图）	数据包（burp抓包 f12的网络里面的请求和返回包）	大概描述（业务等，最好以业务顺序排序）

作业3（对目标做被动和主动的信息收集并整合处理信息成为一份信息收集报告，表现格式和文件格式不限，但是一定要标注出哪些是主要业务、哪些是边缘业务、哪些是新开业务、哪些是外包业务）

下节课预告：

社会工程学的基础知识

社会工程学中的信息收集该如何做

社会工程学途径收集到的信息该如何利用

学习Web安全高级攻防、JAVA代码审计、红队攻防，欢迎加入“MS08067安全实验室”（www.ms08067.com）

Ms08067